

DIAGNOSTIC LAB SaaS

Complete Product, Business, Workflow & Technical Blueprint

A reusable platform for diagnostic laboratories covering online test booking, home sample collection, phlebotomist operations, sample tracking, laboratory processing, report approval and digital report delivery.

Document	Master Product Blueprint
Version	v1.0
Primary model	B2B SaaS / White-label diagnostic lab platform
Recommended starting architecture	MEAN + modular monolith + multi-tenancy

Purpose of this document

Use this document as the master reference before starting implementation. The goal is to understand the entire product first, then convert the blueprint into database design, APIs, UI screens, development tasks and deployment steps.

Table of Contents

- 1. Product vision and positioning
- 2. What the platform actually contains
- 3. Complete end-to-end lifecycle
- 4. User roles and responsibilities
- 5. Patient portal
- 6. Lab admin panel
- 7. Phlebotomist application
- 8. Sample lifecycle and tracking
- 9. Laboratory processing and report lifecycle
- 10. Test catalogue and pricing
- 11. Multi-tenant SaaS architecture
- 12. Recommended MERN technical architecture
- 13. Database/domain model
- 14. Authentication, RBAC and security
- 15. Payments, notifications and integrations
- 16. Healthcare-specific risks and operational challenges
- 17. AI opportunities
- 18. MVP and phased roadmap
- 19. Scalability strategy
- 20. Suggested development order
- 21. Pre-launch checklist
- 22. Final architecture summary

1. Product Vision and Positioning

Core idea: Build a complete digital operating system for independent diagnostic laboratories. A lab can use the platform to publish its tests, accept online bookings, schedule home collections, assign phlebotomists, track samples, process results, approve reports and deliver reports digitally to patients.

Business model: The same core software can serve many labs. Each lab becomes a tenant with its own patients, staff, tests, pricing, branches, bookings and reports.

Product positioning: "Complete digital operating system for diagnostic laboratories."

The platform has two major sides:

- Patient-facing: test discovery, booking, home collection, payment, tracking and report access.
- Lab-facing: operations, staff, sample lifecycle, result/report management, billing, analytics and administration.

2. What the Platform Actually Contains

Interface / Module	Primary users	Main purpose
Patient Portal	Patients / family members	Book tests, manage appointments, track collection, view reports
Lab Admin Panel	Lab owner / manager	Manage the laboratory operation
Reception / Staff	Front desk	Bookings, patients, payments, support
Phlebotomist App	Home collection staff	Assigned visits, OTP, collection status, sample handover
Technician Workflow	Lab technicians	Sample processing and result entry/import
Pathologist Workflow	Authorized reviewer	Review and approve final reports
Super Admin	Your company	Manage labs, subscriptions and platform configuration

Think in business domains rather than pages. Pages are only the UI representation of business domains.

Core domains

- Identity and access
- Lab and branch management
- Test catalogue and packages
- Patient management
- Booking and appointments
- Home collection
- Phlebotomist management
- Sample tracking
- Laboratory processing
- Results and reports
- Payments and invoices
- Notifications
- Analytics
- Audit and compliance

- SaaS subscription management

3. Complete End-to-End Lifecycle

The most important workflow is the journey from booking to final report.

PATIENT

Select test/package -> Select home collection -> Select date/time -> Enter patient/address -> Payment -> Booking confirmed

LAB

Booking received -> Verify availability -> Assign phlebotomist -> Collection visit scheduled

COLLECTION

Phlebotomist travels -> Arrived -> Patient OTP verified -> Sample collected -> Sample labelled -> Sample handed over / transported

LABORATORY

Sample received -> Sample accepted/rejected -> Processing -> Results entered/imported -> Technical checks -> Authorized review -> Final report approved

DELIVERY

Report generated -> Secure storage -> Patient notification -> Patient views/downloads report

Every important state change should be timestamped and attributable to a user/system process. This creates traceability and makes operational troubleshooting possible.

4. User Roles and Responsibilities

Role	Key permissions
Super Admin	Create/manage tenants, subscriptions, platform settings and support
Lab Admin / Owner	Manage lab, branches, staff, tests, prices, bookings, reports and analytics
Lab Manager	Operational management with restricted administrative access
Receptionist	Patients, bookings, appointments, basic billing/support
Phlebotomist	Assigned home visits, OTP verification, collection status, sample handover
Lab Technician	Sample processing, result entry/import, technical workflow
Pathologist / Authorized Reviewer	Review, approve and publish reports
Accountant	Payments, invoices, refunds and financial reports
Patient	Own profile, bookings, payments, reports and authorized family records

Use RBAC. The backend must enforce authorization; frontend hiding is not a security control.

5. Patient Portal

The patient portal should be simple and mobile-friendly.

Core screens

- Home/search
- Test and package listing
- Test details
- Cart/booking summary
- Patient details

- Family member selection
- Address management
- Date/time slot selection
- Payment
- Booking confirmation
- My bookings
- Collection tracking
- My reports
- Report viewer / secure download
- Profile and notification preferences

Family members are useful because one account may book tests for parents, spouse or children. Keep each patient record distinct while making the account relationship explicit.

Typical journey: select package -> home collection -> date/time -> address -> payment -> confirmation -> phlebotomist assignment -> OTP verification -> sample collection -> report approval -> notification -> secure report access.

6. Lab Admin Panel

The admin panel is the operational heart of the SaaS product.

Dashboard KPIs

- Today's bookings
- Pending collections
- Assigned/unassigned collections
- Samples collected
- Samples awaiting receipt
- Samples processing
- Reports pending review
- Reports published
- Cancelled bookings
- Revenue and payment status

Main modules

- Patients
- Tests
- Packages
- Pricing
- Bookings/orders
- Appointment slots
- Home collection

- Phlebotomists and employees
- Samples
- Results
- Reports
- Payments and invoices
- Coupons/offers
- Branches
- Notifications
- Analytics
- Settings
- Roles and permissions
- Audit logs

The lab should configure its own tests, prices, package combinations, collection charges, operating hours, service areas, report branding and notification templates.

7. Phlebotomist Application

The phlebotomist interface should be optimized for quick actions on a mobile device.

TODAY'S ASSIGNMENTS

09:00 AM - Rahul Sharma - CBC + LFT - [Navigate] [Start]

11:00 AM - Amit Kumar - Thyroid Profile - [Navigate] [Start]

VISIT

Assigned -> On the way -> Arrived -> OTP verified -> Collection started -> Sample collected -> Handover complete

At collection time the app can record arrival time, OTP verification, collection start/end, sample/tube details, collection notes, acknowledgement where appropriate, sample identifier and handover status.

Do not give the phlebotomist broad access to patient history or administrative data unless genuinely required.

8. Sample Lifecycle and Tracking

A booking is not the same thing as a physical specimen. Sample tracking is a core operational domain.

Recommended state machine

BOOKED -> ASSIGNED -> ON THE WAY -> ARRIVED -> OTP VERIFIED -> COLLECTED -> IN TRANSIT -> RECEIVED -> ACCEPTED / REJECTED -> PROCESSING -> RESULT READY -> REVIEWED -> PUBLISHED

Each sample should have a unique identifier. Barcode/QR can later connect the physical tube to the digital order.

Rejection/recollection reasons

- Wrong or missing tube
- Insufficient volume
- Leakage/damaged container

- Incorrect labelling
- Sample quality problem
- Transport delay
- Sample beyond allowed processing window
- Patient unavailable for recollection

A rejection should create a structured event with reason, person, time and next action.

9. Laboratory Processing and Report Lifecycle

A report is a controlled clinical artifact. Separate result entry from final authorization.

Recommended workflow

Sample received -> Processing started -> Results entered/imported -> Technical validation -> Authorized reviewer review -> Approved -> Report generated -> Published -> Patient notified

Ordinary staff should not silently change finalized reports. Use controlled amendment/reissue workflows with audit history.

Report storage

- Generate the final report document.
- Store large files in secure object storage rather than directly in MongoDB.
- Store report metadata in the database.
- Use authenticated and authorized retrieval.
- Maintain versions/audit events where required.

10. Test Catalogue and Pricing

Tests should be configuration-driven, not hard-coded.

Field	Purpose
Test ID / Code	Unique internal identifier
Name	CBC, TSH, HbA1c, etc.
Category	Haematology, Biochemistry, Thyroid, etc.
Sample type	Blood, urine, etc.
Preparation	Fasting/non-fasting and instructions
TAT	Expected reporting turnaround
Price	Lab-specific selling price
Reference information	Ranges/instructions configured by lab
Status	Active/inactive

Packages combine tests with package-level pricing. Keep underlying tests independent so packages can change without duplicating test definitions.

For multi-lab operation, pricing should be tenant-specific. Lab A and Lab B can sell the same test at different prices.

11. Multi-Tenant SaaS Architecture

If you plan to sell the software to many laboratories, build for multi-tenancy from the beginning.

```
YOUR PLATFORM
|-- Lab A tenant
| |-- Patients / Staff / Tests & prices / Bookings / Samples / Reports
|-- Lab B tenant
| |-- Patients / Staff / Tests & prices / Bookings / Samples / Reports
```

Every tenant-owned record should carry a tenant/lab identifier. Backend authorization should derive tenant context from the authenticated session and enforce isolation. Never trust a tenant ID blindly from the browser.

White-label options

- labA.yourplatform.com
- labB.yourplatform.com
- Custom domains for higher-tier clients
- Lab-specific logo, colors and report branding
- Lab-specific pricing, services, staff and branches

Recommended starting strategy: shared application and managed database cluster with strict tenant isolation. At larger scale, high-volume tenants can later move to dedicated databases/infrastructure if needed.

12. Recommended MERN Technical Architecture

MERN is a practical starting stack for this product.

```
FRONTEND
React / Next.js or React SPA
|-- Patient Portal
|-- Lab Admin
|-- Phlebotomist UI

BACKEND
Node.js + Express
|-- Auth
|-- Tenant
|-- Patient
|-- Catalogue
|-- Booking
|-- Collection
|-- Sample
|-- Result/Report
|-- Payment
|-- Notification
|-- Audit

DATA / INFRASTRUCTURE
MongoDB -> application data
Object storage -> reports
Redis / queues -> jobs/cache when needed
External APIs -> payment, SMS, WhatsApp, email, maps
```

Start as a modular monolith. Keep strong business-module boundaries inside one backend. Extract services later only when scale, reliability or team boundaries justify it.

Avoid microservices just because the product is expected to scale. Good module boundaries give you a path to scale with less complexity.

13. Database / Domain Model

Conceptual collection set:

Domain	Candidate collections/entities
Identity	users, roles, permissions, sessions
Tenancy	labs, branches, subscriptions, tenantSettings
Catalogue	tests, testParameters, packages, pricing
Patients	patients, patientAddresses, familyMembers
Orders	orders, appointments, orderItems, invoices, payments
Collection	collectionRequests, assignments, otpEvents
Samples	samples, sampleEvents, rejectionEvents
Laboratory	processingRuns, results, resultVersions
Reports	reports, reportVersions, reportAccessEvents
Communication	notifications, templates, deliveryEvents
Operations	staffSchedules, serviceAreas, supportTickets
Audit	auditLogs

Important relationships: an order can contain multiple tests; a package contains multiple tests; one collection can produce multiple physical samples; a report can represent multiple results. Model these explicitly.

Index likely high-use fields such as tenantId, patientId, orderId, sampleId, status, scheduledAt and reportId. Final indexes should be driven by actual query patterns.

14. Authentication, RBAC and Security

Healthcare software requires a higher security standard than a brochure or basic e-commerce site.

Minimum controls

- Secure authentication/session management
- Strong password handling if passwords are used
- OTP expiry and rate limiting
- API-layer RBAC
- Strict tenant isolation
- HTTPS
- Secure report/file storage
- Audit logs
- Input validation
- Rate limiting and abuse protection
- Backups and recovery

- Least-privilege access
- Secrets stored outside source code

Audit examples

Who viewed a report? Who approved it? Who changed patient details? Who cancelled a booking? Who rejected a sample? Who changed a price? Who published a report? These actions should be attributable and timestamped.

Healthcare/privacy note

Before commercial launch, have the actual operating model reviewed for applicable Indian healthcare, privacy, contractual and regulatory requirements. If using ABDM or other interoperability ecosystems, design consent and security requirements into the architecture.

15. Payments, Notifications and Integrations

External services should be hidden behind provider interfaces so a vendor can be replaced later.

Integration	Use
Payment gateway	Booking payments, refunds, payment status
SMS	OTP and transactional notifications
WhatsApp	Reminders/status/report notifications where appropriate
Email	Receipts and notifications
Maps/geocoding	Address and collection routing
Object storage	Report PDFs/documents
Barcode/QR	Sample identification
LIS/instruments	Future result exchange
ABDM	Future interoperability where appropriate
Accounting	Future finance reconciliation

Use queues/background jobs for notifications, document generation, reminders and other work that should not block the main booking request.

16. Healthcare-Specific Risks and Operational Challenges

Challenge	Design response
Wrong patient/sample	Unique IDs, verification, labels/barcodes and controlled workflow
Incorrect result	Validation, permissions, review/approval, versioning and audit
Sample rejection	Explicit reason and recollection workflow
Missed collection	Status model, reminders, reassignment and support
Late transport	Timestamps, SLA/TAT tracking and dashboard
Tenant data leakage	Tenant isolation on every data-access path
Unauthorized report access	Authenticated/authorized secure file delivery
Staff misuse	Least privilege, audit logs and session controls
Vendor outage	Provider abstraction, retries, queues and visible failures

Challenge	Design response
Growth	Indexes, queues, object storage, caching and eventual service extraction

Do not delete historical clinical/operational data simply to fix a workflow mistake. Use corrections, versions, cancellation states and audit events.

17. AI Opportunities

Introduce AI after the core workflow is reliable. Operational correctness comes first.

AI feature	Potential value	Guardrail
Patient assistant	Booking/preparation questions using lab-approved information	Don't invent medical instructions
Report explanation	Explain terminology and values simply	Clearly distinguish explanation from diagnosis
Operations assistant	Questions about pending reports, collections and revenue	Restrict to authorized tenant data
Delay analysis	Find patterns in collection/sample delays	Use auditable source data
Demand forecasting	Predict demand	Planning support, not certainty
Route optimization	Improve phlebotomist scheduling	Human override

AI should never silently modify clinical results. Clinical data entry, review and publication remain controlled workflows.

18. MVP and Phased Roadmap

Build the smallest complete business loop that a real laboratory can use.

Phase 1 - MVP

- Lab onboarding
- Lab admin login
- Patient registration
- Test catalogue
- Test/package pricing
- Online booking
- Appointment slots
- Home collection
- Phlebotomist assignment
- OTP verification
- Sample tracking
- Report upload/generation
- Report approval
- Patient report access
- Basic notifications
- Basic audit logs

Phase 2 - Commercial

- Online payments
- Coupons/offers
- Family members
- Multiple branches
- Employee scheduling
- Invoices
- WhatsApp/SMS automation
- Barcode/QR
- Better dashboards
- Lab subscription billing

Phase 3 - Advanced operations

- LIS integration
- Instrument/result integration
- Advanced analytics
- Doctor/referral management
- Accounting integration
- Route planning
- ABDM/interoperability where appropriate

Phase 4 - AI

- AI patient assistant
- AI report explanation
- AI operations analytics
- Demand forecasting
- Route optimization

19. Scalability Strategy

Scale the architecture progressively.

STAGE 1

Modular backend -> managed MongoDB -> object storage -> frontend -> background worker as needed

STAGE 2

Redis/cache -> queues -> CDN -> horizontal backend scaling -> observability

STAGE 3

Separate high-load workloads such as notifications, report generation, search or analytics if needed

STAGE 4

Extract independent services only when justified

Scalability includes more than server capacity: tenant isolation, indexing, asynchronous work, storage, monitoring, backups, recovery and predictable deployment are equally important.

20. Suggested Development Order

- 1 Finalize business requirements and user roles.
- 2 Define tenant model and authorization strategy.
- 3 Define domain model and database collections.
- 4 Define status/state machines for booking, collection, sample and report.
- 5 Define API contracts.
- 6 Build authentication and RBAC.
- 7 Build lab/tenant setup.
- 8 Build test catalogue and packages.
- 9 Build patient and address management.
- 10 Build booking and appointment slots.
- 11 Build home collection assignment.
- 12 Build phlebotomist workflow and OTP.
- 13 Build sample tracking.
- 14 Build result/report workflow.
- 15 Build patient report access.
- 16 Add notifications.
- 17 Add payments and invoices.
- 18 Add audit logs and operational analytics.
- 19 Add multi-branch and SaaS features.
- 20 Harden security, monitoring, backups and deployment.
- 21 Pilot with one real laboratory before broad rollout.

This sequence keeps the backend business model ahead of the UI and ensures the booking-to-report loop works before advanced features are added.

21. Pre-Launch Checklist

Product

- Patient can complete a booking
- Lab can assign/reschedule collection
- Phlebotomist can complete OTP workflow
- Lab can receive and track a sample
- Authorized person can approve a report
- Patient can securely view final report

Security

- Tenant isolation tested

- RBAC tested
- Report access tested
- OTP abuse/rate limiting tested
- Secrets removed from source
- HTTPS enabled
- Backups configured
- Audit logs verified

Operations

- Sample rejection workflow
- Failed collection workflow
- Payment failure workflow
- Notification retry/failure visibility
- Support can investigate an order
- Report correction workflow

Business

- Pricing model
- Subscription/contract model
- Lab onboarding process
- Support process
- Data ownership/responsibilities documented
- Applicable legal/privacy review

22. Final Architecture Summary

The product is a multi-tenant healthcare SaaS platform with three primary operational interfaces: Patient Portal, Lab Admin and Phlebotomist App, supported by technician/pathologist workflows and a Super Admin console.

PATIENT

Booking -> Payment -> Appointment -> Collection -> Sample tracking -> Report delivery

LAB

Catalogue -> Booking -> Staff assignment -> Sample receipt -> Processing -> Review -> Report -> Analytics

PLATFORM

Multi-tenancy -> Authentication/RBAC -> Audit -> Notifications -> Storage -> Payments -> Integrations -> Monitoring

Recommended initial technology direction: MERN, modular monolith, multi-tenant data model, managed MongoDB, secure object storage for reports, background jobs/queues when needed, and provider abstraction for payments/notifications.

The architecture is a foundation, not a frozen design. Mobile apps, barcode tracking, multiple branches, LIS integration, ABDM interoperability and AI can be added as modules/integrations without replacing the core architecture.

Next technical step

After reviewing this document, convert it into a detailed technical specification: exact MongoDB schemas, relationships, API endpoints, authentication flow, status enums/state machines, MERN folder structure, UI screen map and development sprint plan. Do that before writing most production code.

Planning note: This document is a product/technical blueprint, not legal or medical advice. Healthcare/privacy/compliance requirements should be validated for the actual services, data flows, jurisdictions and integrations before commercial launch.

DIAGNOSTIC LAB SaaS

Detailed Technical Specification, Architecture & Build Plan

Production-oriented implementation document for turning the existing Product & Technical Blueprint into code.

Document	Technical Specification & Architecture
Version	v1.0 — implementation baseline
Primary architecture	MERN + TypeScript + modular monolith + multi-tenancy
Primary interfaces	Patient Portal + Lab Admin + Phlebotomist + Technician/Reviewer + Super Admin
Primary database	MongoDB
Files	Private object storage (S3-compatible)
Async processing	Redis + BullMQ worker

Source basis: this document expands the uploaded “Diagnostic Lab SaaS — Product & Technical Blueprint”, whose stated starting direction is MERN + modular monolith + multi-tenancy and whose next technical step is exact schemas, relationships, APIs, authentication, state machines, folder structure, UI map and sprint plan. ■filecite■turn0file0■L568-L574■

Important: this is an engineering blueprint, not legal, medical, or regulatory advice. The source document specifically calls for applicable Indian healthcare/privacy/compliance requirements to be validated before commercial launch.

■filecite■turn0file0■L361-L364■

1. Architecture Decision — Build This First

The safest starting point is a **modular monolith**: one backend deployment, one primary MongoDB cluster, one worker process when needed, but strict domain boundaries in code. This follows the uploaded blueprint's recommendation to avoid premature microservices while preserving a path to later extraction. ■filecite■turn0file0■L284-L310■

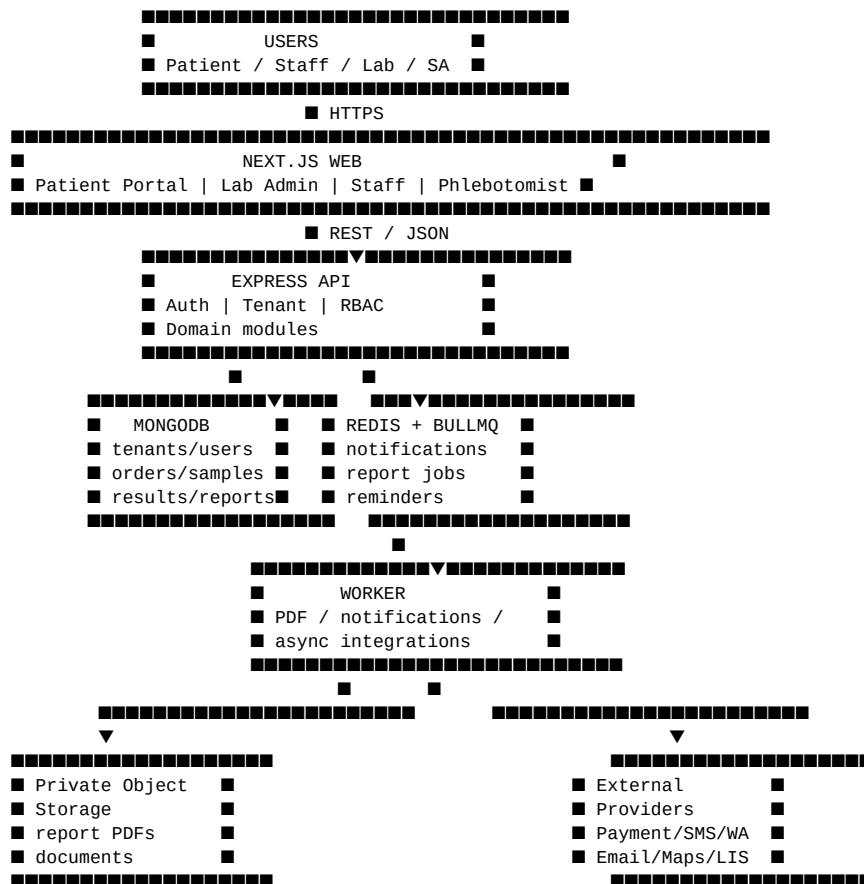
Recommended implementation stack

Layer	Technology	Decision
Web	Next.js + React + TypeScript	Use one web codebase with route groups for patient, lab, staff and platform surfaces.
UI	Tailwind CSS + component library	Consistent responsive admin and mobile-first collection UI.
API	Node.js + Express + TypeScript	REST API, modular domain boundaries.
Validation	Zod	Validate request body, params, query and selected configuration.
Database	MongoDB Atlas + Mongoose	Tenant-owned documents include tenantId; indexes follow real query patterns.
Cache/queues	Redis + BullMQ	Notifications, report jobs, reminders and other non-blocking work.
File storage	S3-compatible private bucket	Report PDFs/documents; DB stores metadata, not large report files.
Auth	Short-lived access token + rotating refresh tokens + httpOnly cookies	REST only strategy; cookies for browser sessions.
Payments	Provider adapter; Razorpay is an initial India option	Use a payment provider behind an interface so it can be replaced.
Messaging	Provider adapters for SMS/WhatsApp/email	All outbound communication goes through a notification module.
Maps	Provider adapter	Geocoding and collection routing.
Observability	Structured logs + error tracking + metrics	Add before pilot; health endpoints from day one.
Testing	Vitest/Jest + Supertest + Playwright	Unit, API integration and critical end-to-end workflows.
CI/CD	GitHub Actions	Lint, typecheck, tests, build, migration/index checks and deployment.

Core principle: business domains are the architecture. Pages/controllers are only delivery mechanisms. The source blueprint explicitly lists identity, tenancy, catalogue, patients, booking, collection, phlebotomists, samples, laboratory processing, reports, payments, notifications, analytics and audit as core domains. ■filecite■turn0file0■L66-L87■

2. Complete System Architecture

Logical architecture



Deployment rule

- Frontend can be deployed separately from the API, but all sensitive operations remain server-side.
- The browser never decides tenantId, role permissions, report access, or sample state transitions.
- The API derives tenant context from the authenticated session and enforces isolation on every tenant-owned query, matching the source blueprint. ■filecite■turn0file0■L261-L283■
- Private reports are served through an authorization check and short-lived signed retrieval mechanism, never a public bucket URL.
- Worker jobs are idempotent and retryable; booking confirmation should not wait for SMS/email/report generation.

3. Domain / Module Architecture

Module	Owns	Must not own
identity	users, sessions, roles, permissions	business order state
tenancy	labs, branches, tenant settings, subscription	patient clinical results
catalogue	tests, parameters, packages, tenant pricing	payments
patients	patient records, addresses, family relationships	sample processing
orders	orders, order items, appointment lifecycle, cancellation	final report authorization
collection	collection requests, slots, assignments, OTP events	lab result values
samples	physical specimens, identifiers, sample events, rejection/recollection	payment settlement
laboratory	processing runs, results, result versions, technical validation	patient login
reports	report versions, approval/publication, access events	booking pricing
billing	payments, invoices, refunds, coupons	clinical result approval
notifications	templates, notification jobs, delivery events	tenant authorization
analytics	operational KPIs and read models	source-of-truth clinical data
audit	immutable audit events	business mutation logic
platform-admin	tenant provisioning, subscription, support access	normal tenant business operations

Module rules

- Each module has controller/route, application service, domain rules, repository/data access and schemas.
- Modules communicate through service interfaces and explicit events rather than importing another module's private models.
- Cross-domain writes are coordinated by an application service or transaction/outbox pattern where required.
- Clinical/report finalization is treated as a controlled workflow; ordinary staff cannot silently mutate a published report.

filecite turn0file0L229-L241

4. Multi-Tenancy Design

Tenant isolation is a first-class security boundary. The source blueprint requires every tenant-owned record to carry a tenant/lab identifier and says the backend must derive tenant context from the authenticated session rather than trusting the browser. ■filecite■turn0file0■L261-L270■

Recommended model: shared MongoDB cluster, shared collections, mandatory tenantId on tenant-owned documents.

```
Request
↓
Authentication middleware
↓
session.userId → User lookup → user.tenantId / platform role
↓
TenantContext { tenantId, branchId?, userId, role }
↓
Authorization middleware
↓
Repository requires tenantId for every tenant-owned query
```

Never allow

- req.body.tenantId to determine data access.
- A frontend route parameter to bypass tenant authorization.
- A repository method such as findById(id) for tenant-owned data without tenant scope.
- Public object-storage URLs for patient reports.

Repository pattern

```
interface TenantScopedRepository<T> {
  findById(ctx: TenantContext, id: string): Promise<T | null>;
  findMany(ctx: TenantContext, filter: Filter<T>): Promise<T[]>;
  create(ctx: TenantContext, input: CreateInput<T>): Promise<T>;
  update(ctx: TenantContext, id: string, input: UpdateInput<T>): Promise<T>;
}
```

Scale path

Start shared. If a high-volume tenant eventually requires dedicated infrastructure, the domain layer should remain unchanged while the tenancy/data-access layer can route that tenant to a dedicated database. This follows the source blueprint's staged tenancy strategy. ■filecite■turn0file0■L281-L283■

5. MongoDB Domain Model

The uploaded blueprint proposes identity, tenancy, catalogue, patient, order, collection, sample, laboratory, report, communication, operations and audit collections. ■filecite■turn0file0■L316-L335■ The following turns that conceptual list into implementation-ready schemas.

Core collections

Collection	Key fields	Critical indexes
labs	_id, slug, name, status, branding, serviceAreas	slug unique, status
branches	tenantId, name, address, hours, status	tenantId+status
users	tenantId, branchIds, roleIds, name, phone, email, status	tenantId+phone, tenantId+email
sessions	userId, tenantId, tokenHash, expiresAt, revokedAt	userId, expiresAt TTL
roles	tenantId, name, permissions	tenantId+name unique
tests	tenantId, code, name, category, sampleTypes, prep, tat, active	tenantId+code unique
packages	tenantId, code, name, testIds, price, active	tenantId+code unique
patients	tenantId, name, dob, sex, phone, identifiers	tenantId+phone, tenantId+search fields
patientAddresses	tenantId, patientId, address, geo, label	tenantId+patientId
orders	tenantId, patientId, items, status, totals, paymentStatus	tenantId+createdAt, tenantId+patientId, tenantId+status
appointments	tenantId, orderId, branchId, scheduledAt, status	tenantId+scheduledAt, tenantId+status
collectionRequests	tenantId, orderId, patientId, addressId, slot, status	tenantId+scheduledAt, tenantId+status
collectionAssignments	tenantId, collectionRequestId, phlebotomistId, status	tenantId+phlebotomistId+scheduledAt
otpEvents	tenantId, collectionId, purpose, hash, expiresAt, verifiedAt	collectionId+purpose, expiresAt TTL
samples	tenantId, orderId, sampleCode, type, status, collectedAt, receivedAt	tenantId+sampleCode unique, tenantId+status
sampleEvents	tenantId, sampleId, type, actorId, at, metadata	tenantId+sampleId+at
processingRuns	tenantId, sampleIds, startedAt, completedAt, status	tenantId+status
results	tenantId, sampleId, testId, parameterValues, status	tenantId+sampleId, tenantId+testId
resultVersions	tenantId, resultId, version, values, changedBy, reason	resultId+version unique
reports	tenantId, orderId, status, currentVersionId, publishedAt	tenantId+orderId unique
reportVersions	tenantId, reportId, version, storageKey, hash, approvedBy	reportId+version unique
reportAccessEvents	tenantId, reportId, patientId, actorId, action, at	tenantId+reportId+at
payments	tenantId, orderId, provider, providerPaymentId, status, amount	tenantId+orderId, provider+providerPaymentId unique
invoices	tenantId, orderId, invoiceNo, totals, status	tenantId+invoiceNo unique
notifications	tenantId, recipient, channel, template, status, attempts	tenantId+status+scheduledAt
auditLogs	tenantId, actorId, action, entityType, entityId, before, after, at	tenantId+entityType+entityId+at

6. Exact Schema Baselines

Use Mongoose schemas with TypeScript types. The examples below define the minimum production shape; add domain-specific fields only when a real workflow requires them.

Tenant / User

```
Lab {
  _id: ObjectId,
  slug: string,                // unique platform-wide
  name: string,
  legalName?: string,
  status: "ACTIVE" | "SUSPENDED" | "PENDING",
  branding: {
    logoKey?: string,
    primaryColor?: string,
    reportFooter?: string
  },
  settings: {
    timezone: string,
    currency: "INR",
    bookingPolicy: {...},
    reportPolicy: {...}
  },
  createdAt, updatedAt
}

User {
  _id: ObjectId,
  tenantId?: ObjectId,         // absent only for platform-level user
  branchIds: ObjectId[],
  roleIds: ObjectId[],
  name: string,
  phone?: string,
  email?: string,
  passwordHash?: string,
  status: "ACTIVE" | "INVITED" | "SUSPENDED",
  lastLoginAt?: Date,
  createdAt, updatedAt
}
```

Test / Package / Pricing

```
Test {
  tenantId: ObjectId,
  code: string,
  name: string,
  category: string,
  sampleTypes: ["BLOOD", "URINE", ...],
  preparation?: string,
  tatMinutes?: number,
  referenceInfo?: [{ parameterId, text }],
  active: boolean
}

TestParameter {
  tenantId: ObjectId,
  testId: ObjectId,
  code: string,
  name: string,
  unit?: string,
  datatype: "NUMBER" | "TEXT" | "SELECT" | "BOOLEAN",
  referenceRange?: {...},
  criticalRules?: [...]
}

Package {
  tenantId: ObjectId,
  code: string,
  name: string,
  testIds: ObjectId[],
  price: number,
  active: boolean
}
```

Order / Collection / Sample

```
Order {
  tenantId: ObjectId,
```

```

orderNo: string,
patientId: ObjectId,
items: [{
  kind: "TEST" | "PACKAGE",
  catalogueId: ObjectId,
  nameSnapshot: string,
  priceSnapshot: number,
  quantity: number
}],
appointmentId?: ObjectId,
collectionRequestId?: ObjectId,
subtotal: number,
collectionFee: number,
discount: number,
tax: number,
total: number,
paymentStatus: "PENDING" | "PAID" | "FAILED" | "REFUNDED" | "PARTIAL",
status: "DRAFT" | "CONFIRMED" | "IN_PROGRESS" | "COMPLETED" |
  "CANCELLED" | "REFUND_PENDING",
createdAt, updatedAt
}

Sample {
  tenantId: ObjectId,
  orderId: ObjectId,
  patientId: ObjectId,
  sampleCode: string,
  specimenType: string,
  tubeType?: string,
  status: SampleStatus,
  collectedAt?: Date,
  receivedAt?: Date,
  rejection?: { reasonCode, note, rejectedAt, rejectedBy },
  createdAt, updatedAt
}

```

7. State Machines — Do Not Implement as Free-Form Strings

The source blueprint explicitly defines a sample lifecycle and report lifecycle. These states should be centralized in domain code so invalid transitions fail server-side. ■filecite■turn0file0■L206-L235■

Booking

DRAFT → CONFIRMED → SCHEDULED → COLLECTION_IN_PROGRESS → SAMPLE_RECEIVED
→ PROCESSING → REPORT_READY → COMPLETED

Allowed side transitions:

CONFIRMED → CANCELLED

SCHEDULED → CANCELLED

SCHEDULED → RESCHEDULED → SCHEDULED

COLLECTION_IN_PROGRESS → COLLECTION_FAILED → RESCHEDULED

Collection visit

ASSIGNED → ON_THE_WAY → ARRIVED → OTP_VERIFIED
→ COLLECTION_STARTED → COLLECTED → HANDOVER_COMPLETE

Failure:

ASSIGNED / ON_THE_WAY / ARRIVED → FAILED

FAILED → RESCHEDULED

Sample

BOOKED → ASSIGNED → ON_THE_WAY → ARRIVED → OTP_VERIFIED
→ COLLECTED → IN_TRANSIT → RECEIVED
→ ACCEPTED → PROCESSING → RESULT_READY → REVIEWED → PUBLISHED

RECEIVED → REJECTED → RECOLLECTION_REQUIRED → COLLECTED

Report

DRAFT → GENERATED → PENDING_REVIEW → APPROVED → PUBLISHED

PUBLISHED → AMENDMENT_REQUESTED → AMENDED → REAPPROVED → REPUBLISHED

Transition API rule

```
transitionSample({
  tenantId,
  sampleId,
  action: "RECEIVE"
})
```

→ load current state
→ verify action is allowed
→ verify actor permission
→ update state
→ append SampleEvent
→ append AuditLog
→ publish domain event if needed

8. REST API Contract

Base path: **/api/v1**. Use consistent response envelopes, validation errors and pagination. Controllers should stay thin; business rules belong in application/domain services.

Authentication

Method	Endpoint	Purpose	Auth
POST	/auth/login	Staff login	Public
POST	/auth/refresh	Refresh session	Session
POST	/auth/logout	Revoke session	Session
GET	/auth/me	Current user + tenant context	Session
POST	/auth/patient/request-otp	Patient OTP login/request	Public/rate-limited
POST	/auth/patient/verify-otp	Patient OTP verification	Public/rate-limited

Catalogue / Patients / Booking

Method	Endpoint	Purpose
GET	/catalogue/tests	List active tests
POST	/catalogue/tests	Create test
PATCH	/catalogue/tests/:id	Update test
GET	/catalogue/packages	List packages
POST	/catalogue/packages	Create package
GET	/patients	Search/list patients
POST	/patients	Create patient
GET	/patients/:id	Patient details
POST	/orders	Create booking/order
GET	/orders/:id	Order detail
POST	/orders/:id/cancel	Cancel order
GET	/appointments/slots	Available slots
POST	/appointments	Book appointment

Collection / Sample / Laboratory / Reports

Method	Endpoint	Purpose
GET	/collections	Operational collection queue
POST	/collections/:id/assign	Assign phlebotomist
POST	/collections/:id/start	Start visit
POST	/collections/:id/verify-otp	Verify patient OTP
POST	/collections/:id/collect	Complete collection
POST	/collections/:id/fail	Record failed collection
GET	/samples	Sample queue
POST	/samples/:id/receive	Receive sample
POST	/samples/:id/reject	Reject sample
POST	/samples/:id/process	Start processing
POST	/results/:id/submit	Submit results
POST	/reports/:id/review	Reviewer decision
POST	/reports/:id/publish	Publish final report

Method	Endpoint	Purpose
GET	/reports/:id	Authorized report metadata
GET	/reports/:id/download	Authorized secure retrieval

9. API Standards, Validation & Errors

Response envelope

```
Success:
{
  "success": true,
  "data": {...},
  "meta": { "requestId": "...", "page": 1, "limit": 20, "total": 120 }
}

Error:
{
  "success": false,
  "error": {
    "code": "SAMPLE_INVALID_TRANSITION",
    "message": "Sample cannot be published from RECEIVED state",
    "fields": {}
  },
  "meta": { "requestId": "..."}
}
```

Validation

- Zod schemas for every externally supplied body, query and path parameter.
- Reject unknown fields for sensitive mutation endpoints where practical.
- Normalize phone/email before lookup and enforce tenant-scoped uniqueness.
- Never log passwords, OTPs, access tokens, report contents or unnecessary patient data.

HTTP status convention

Status	Use
200	Successful read/update/action
201	Created resource
400	Malformed/invalid request
401	Not authenticated
403	Authenticated but not authorized
404	Resource not found within tenant scope
409	Conflict/state/version/uniqueness
422	Business validation failure
429	Rate limit exceeded
500	Unexpected server error
503	Dependency unavailable

The source blueprint calls for input validation, rate limiting, tenant isolation, secure authentication/session management, secure report storage, audit logs, HTTPS, backups and least privilege. ■filecite■turn0file0■L336-L360■

10. Authentication & RBAC

Roles from the source blueprint

Role	Primary permissions
Super Admin	Tenants, subscriptions, platform configuration and support
Lab Admin / Owner	Lab, branches, staff, tests, pricing, bookings, reports, analytics
Lab Manager	Operational management with restricted administration
Receptionist	Patients, bookings, appointments, basic billing/support
Phlebotomist	Assigned visits, OTP, collection status, sample handover
Lab Technician	Sample processing, result entry/import, technical workflow
Pathologist / Reviewer	Review, approve and publish reports
Accountant	Payments, invoices, refunds, financial reports
Patient	Own profile, bookings, payments, reports and authorized family records

Permission format

```
catalogue:test:read
catalogue:test:write
booking:read
booking:write
collection:assign
collection:operate
sample:receive
sample:reject
result:write
report:review
report:approve
report:publish
payment:read
payment:refund
audit:read
tenant:settings
```

Authorization middleware

```
requireAuth()
requireTenantContext()
requirePermission("report:publish")
requireBranchScopeIfApplicable()
→ controller
→ service
```

Critical rule: frontend route guards are for UX only. The backend must enforce authorization. This is explicitly required by the source blueprint. ■filecite■turn0file0■L109-L120■

11. Clinical Data, Reports & Audit Design

A report is a controlled clinical artifact. Keep result entry separate from authorization and publication. The source blueprint requires controlled amendment/reissue workflows and audit history for finalized reports. ■filecite■turn0file0■L229-L241■

Report storage architecture

```
MongoDB:
  report metadata
  status
  order/patient references
  current version
  approval metadata
  hashes
  audit references

Object storage:
  private/restricted/report/{tenantId}/{reportId}/{version}.pdf
```

Secure download sequence

- 1 Authenticate requester.
- 2 Resolve tenant context.
- 3 Load report using tenantId + reportId.
- 4 Verify patient ownership or staff permission.
- 5 Record report access event.
- 6 Generate short-lived signed object-storage URL or stream through an authorized endpoint.
- 7 Never expose storage credentials to the browser.

Audit events

- Patient viewed report.
- Reviewer approved report.
- Staff changed patient demographic information.
- Booking was cancelled/rescheduled.
- Sample was rejected and why.
- Price/catalogue configuration changed.
- Report was published/amended/reissued.

The source blueprint emphasizes attributable, timestamped state changes and asks exactly these types of audit questions. ■filecite■turn0file0■L107-L108■ ■filecite■turn0file0■L357-L360■

12. Redis, Queues & Background Jobs

Do not make the booking request wait for external notifications or document generation. The source blueprint specifically recommends queues/background jobs for notifications, document generation, reminders and other work that should not block booking. ■filecite■turn0file0■L365-L379■

Queue	Jobs	Retry policy
notifications	SMS, email, WhatsApp	Exponential retry + dead-letter after limit
reports	Generate PDF, upload, checksum	Retry if storage/rendering failure
reminders	Upcoming collection/report notifications	Scheduled jobs; idempotent
integrations	Payment/webhook reconciliation, future LIS	Retry + visible failure
analytics	Aggregate operational metrics	Eventually consistent

Job rules

- Every job gets a deterministic idempotency key.
- Workers must tolerate duplicate delivery.
- Persist provider message IDs and status.
- Use dead-letter handling and an admin-visible failure queue.
- Do not put raw patient/report content in Redis unless strictly necessary.

13. Integration Architecture

The source blueprint recommends hiding external vendors behind provider interfaces so vendors can be replaced later.

■filecite■turn0file0■L365-L377■

Provider interfaces

```
PaymentProvider
  createOrder()
  verifyPayment()
  refund()
  parseWebhook()

SmsProvider
  sendOtp()
  sendTransactional()

WhatsAppProvider
  sendTemplate()

EmailProvider
  sendTransactional()

StorageProvider
  putPrivateObject()
  getSignedUrl()
  deleteObject()

MapsProvider
  geocode()
  distanceMatrix()

LISProvider
  importResults()
  exportOrders()           // future

InteroperabilityProvider
  publish/receive(...) // future ABDM/interoperability
```

Initial integration priorities

- 1 Payment gateway for paid online booking.
- 2 SMS OTP and transactional messages.
- 3 Email for receipts/report notifications.
- 4 WhatsApp where operationally appropriate.
- 5 Object storage for reports.
- 6 Maps/geocoding for collection address/routing.
- 7 Barcode/QR for physical sample identification.
- 8 LIS/instrument/ABDM integrations only after core workflow is stable.

14. MERN + TypeScript Folder Structure

```
diagnostic-lab-saas/
├── apps/
│   ├── web/
│   │   ├── app/
│   │   │   ├── (patient)/
│   │   │   ├── (lab)/
│   │   │   ├── (staff)/
│   │   │   ├── (platform)/
│   │   │   ├── api-proxy/ # optional
│   │   │   ├── components/
│   │   │   ├── features/
│   │   │   ├── lib/
│   │   │   ├── hooks/
│   │   │   └── types/
│   │   └──
│   └── api/
│       ├── src/
│       │   ├── config/
│       │   ├── middleware/
│       │   ├── shared/
│       │   │   ├── errors/
│       │   │   ├── http/
│       │   │   ├── logger/
│       │   │   ├── validation/
│       │   │   ├── utils/
│       │   │   └── modules/
│       │   │       ├── auth/
│       │   │       ├── tenancy/
│       │   │       ├── catalogue/
│       │   │       ├── patients/
│       │   │       ├── orders/
│       │   │       ├── collection/
│       │   │       ├── samples/
│       │   │       ├── laboratory/
│       │   │       ├── reports/
│       │   │       ├── billing/
│       │   │       ├── notifications/
│       │   │       ├── analytics/
│       │   │       └── audit/
│       │   └── server.ts
│       └──
├── workers/
│   ├── src/
│   │   ├── queues/
│   │   ├── jobs/
│   │   └── worker.ts
│   └──
├── packages/
│   ├── shared-types/
│   ├── validation/
│   ├── config/
│   └── ui/
│   └──
├── infra/
│   ├── docker/
│   ├── github/
│   └── scripts/
│   └──
├── docs/
│   ├── architecture/
│   ├── api/
│   └── runbooks/
│   └──
├── package.json
├── pnpm-workspace.yaml
└── README.md
```

Inside each backend module

```
modules/samples/
├── sample.routes.ts
├── sample.controller.ts
├── sample.service.ts
├── sample.repository.ts
├── sample.model.ts
├── sample.schema.ts
├── sample.types.ts
└── sample.state-machine.ts
```

```
■■■ sample.events.ts
■■■ sample.test.ts
```

This keeps the product modular without creating network boundaries prematurely.

15. UI / Screen Map

The source blueprint defines the patient portal, lab admin, phlebotomist, technician/reviewer and super-admin surfaces.

■filecite■turn0file0■L57-L65■

Surface	Routes / screens
Patient	/, /tests, /packages, /cart, /checkout, /booking/:id, /bookings, /collections/:id, /reports, /reports/:id, /profile, /family, /addresses
Lab Admin	/dashboard, /patients, /tests, /packages, /pricing, /orders, /appointments, /collections, /phlebotomists, /samples, /results, /reports, /payments, /branches, /n
Phlebotomist	/today, /visits/:id, /visits/:id/arrive, /visits/:id/otp, /visits/:id/collect, /visits/:id/handover, /history
Technician	/worklist, /samples/:id, /processing/:id, /results/:id
Reviewer	/review-queue, /reports/:id/review, /reports/:id/history
Accountant	/payments, /invoices, /refunds, /finance
Super Admin	/tenants, /tenants/:id, /subscriptions, /platform-users, /support, /system-settings

Patient booking flow

Home → Test/Package → Cart → Patient/Family Member → Address
→ Date/Time Slot → Payment → Confirmation → Track Collection
→ Report Notification → Secure Report Viewer

Lab operational dashboard KPIs should include today's bookings, pending/assigned collections, samples collected/awaiting receipt/processing, reports pending review/published, cancellations and revenue/payment status, matching the source blueprint. ■filecite■turn0file0■L151-L175■

16. Core Coding Patterns

Service example — create order

```
async function createOrder(ctx, input) {
  const patient = await patients.requireOwned(ctx, input.patientId);
  const items = await catalogue.resolveItems(ctx, input.items);

  const totals = pricing.calculate(items, input.collectionFee, input.coupon);

  const order = await orders.create(ctx, {
    patientId: patient._id,
    items: items.map(snapshotItem),
    ...totals,
    status: "CONFIRMED",
    paymentStatus: totals.total === 0 ? "PAID" : "PENDING"
  });

  if (order.paymentStatus === "PENDING") {
    const payment = await payments.createCheckout(ctx, order);
    return { order, payment };
  }

  await events.publish("order.confirmed", { tenantId: ctx.tenantId, orderId: order._id });
  return { order };
}
```

Do not

- Put MongoDB queries directly in React components.
- Allow controllers to contain pricing/state-machine logic.
- Reuse patient-facing DTOs for staff/admin screens without permission filtering.
- Return Mongoose documents directly as API responses.
- Store report PDFs in MongoDB.
- Let AI write or silently modify clinical results.

The source blueprint explicitly states that AI must never silently modify clinical results and that clinical data entry, review and publication remain controlled workflows. ■filecite■turn0file0■L399-L409■

17. Testing Strategy

Layer	Test	Examples
Unit	Pure domain rules	State transitions, pricing, TAT calculation, permissions
Repository	Tenant isolation	Tenant A cannot retrieve Tenant B data
API	Integration	Login, booking, assignment, sample receipt, report publication
Worker	Job behavior	Retry, idempotency, provider failure
E2E	Critical journeys	Patient booking → collection → report; staff report approval
Security	Abuse/isolation	OTP rate limits, IDOR attempts, role escalation

Mandatory security tests

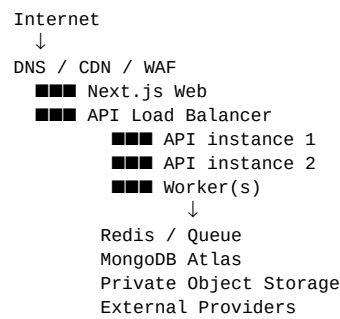
- Change tenantId in request body and verify it has no effect.
- Access another patient's report by changing report ID and verify 404/403.
- Attempt report publication as receptionist/technician and verify denial.
- Replay/duplicate OTP verification and verify safe behavior.
- Submit invalid state transition and verify 409/422.
- Replay payment webhook and verify idempotency.
- Verify audit record for every high-risk mutation.

18. Deployment Architecture

Practical initial environment

Component	Initial deployment
Web	Next.js deployment (Vercel or equivalent)
API	Node/Express container on Render, AWS ECS/Fargate, or equivalent managed container platform
Worker	Separate worker process/container from same codebase
MongoDB	MongoDB Atlas
Redis	Managed Redis
Object storage	Private S3-compatible bucket
DNS/TLS	Cloudflare or cloud DNS + HTTPS
CI/CD	GitHub Actions
Monitoring	Error tracking + structured logs + uptime/health checks

Production topology



The source blueprint's scalability stages are: modular backend + managed MongoDB + object storage; then Redis/cache/queues/CDN/horizontal scaling/observability; then separate high-load workloads; and only then service extraction where justified. [filecite turn0file0L467-L478](#)

19. Environment Variables

```
NODE_ENV=development
PORT=5000
APP_URL=https://app.example.com
API_URL=https://api.example.com
```

```
MONGODB_URI=
REDIS_URL=
```

```
AUTH_ACCESS_SECRET=
AUTH_REFRESH_SECRET=
SESSION_COOKIE_NAME=lab_session
OTP_HASH_SECRET=
```

```
STORAGE_ENDPOINT=
STORAGE_REGION=
STORAGE_BUCKET=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
```

```
PAYMENT_PROVIDER=
PAYMENT_KEY_ID=
PAYMENT_KEY_SECRET=
PAYMENT_WEBHOOK_SECRET=
```

```
SMS_PROVIDER=
SMS_API_KEY=
SMS_SENDER_ID=
```

```
WHATSAPP_PROVIDER=
WHATSAPP_API_KEY=
```

```
EMAIL_PROVIDER=
EMAIL_API_KEY=
EMAIL_FROM=
```

```
MAPS_PROVIDER=
MAPS_API_KEY=
```

```
SENTRY_DSN=
LOG_LEVEL=info
```

Secrets policy

- Never commit .env files containing secrets.
- Use separate secrets for development, staging and production.
- Rotate provider secrets and signing keys using a documented process.
- Do not print secrets in logs or error responses.
- Use secret-management facilities in production rather than plain server files where available.

20. Development Sprint Plan

The source blueprint's development order starts with requirements/roles, tenant model, domain model, state machines and API contracts, then authentication, catalogue, patients, booking, collection, phlebotomist, samples, reports, notifications, payments, analytics, security and pilot. The plan below converts that order into coding sprints.

Sprint	Deliverables	Definition of done
0	Repo, TypeScript, lint, CI, env config, base UI, API health	Developer can clone, run, test and deploy dev build
1	Tenant + users + roles + auth + sessions	RBAC + tenant isolation tests pass
2	Tests + parameters + packages + pricing	Lab can configure catalogue
3	Patients + addresses + family model	Patient CRUD/search and ownership rules work
4	Orders + appointment slots + booking	Patient can complete booking flow
5	Collection requests + assignment + phlebotomist UI + OTP	Visit can reach collected state
6	Samples + events + rejection/recollection	Physical specimen lifecycle traceable
7	Results + processing + reviewer workflow	Authorized reviewer can approve
8	PDF generation + private storage + report access	Patient can securely view final report
9	Notifications + queues + retries	Transactional events delivered asynchronously
10	Payments + invoices + webhook reconciliation	Paid booking/refund flow works
11	Audit + dashboards + support tooling	Operations can investigate issues
12	Security hardening + backups + pilot	Pre-launch checklist passes

21. Day-1 Coding Checklist

- 1 Create a monorepo using pnpm workspaces.
- 2 Create Next.js TypeScript web app.
- 3 Create Express TypeScript API app.
- 4 Create worker package/app.
- 5 Add shared TypeScript types and Zod package.
- 6 Configure ESLint, Prettier, TypeScript strict mode and pre-commit checks.
- 7 Create MongoDB connection module and health endpoint.
- 8 Create environment validation with Zod; fail fast on missing required production variables.
- 9 Create request ID + structured logging middleware.
- 10 Create error class + global error handler.
- 11 Create auth/session module.
- 12 Create TenantContext middleware.
- 13 Create permission middleware.
- 14 Create first Lab/User/Role schemas and seed script.
- 15 Write tenant-isolation and RBAC integration tests before adding business modules.

Suggested initial package commands

```
pnpm create next-app@latest apps/web --ts --eslint --app
mkdir -p apps/api apps/workers packages/shared-types packages/validation
pnpm add -w typescript eslint prettier
pnpm --filter api add express mongoose zod pino cookie-parser helmet cors
pnpm --filter api add bcryptjs jsonwebtoken
pnpm --filter workers add bullmq ioredis
pnpm --filter web add zod
pnpm add -D -w vitest playwright supertest
```

Note: pin versions in package.json/lockfile and upgrade deliberately rather than copying “latest” into production without review.

22. First Production Module — Auth + Tenant

Implement this module completely before booking. Everything else depends on it.

Data flow

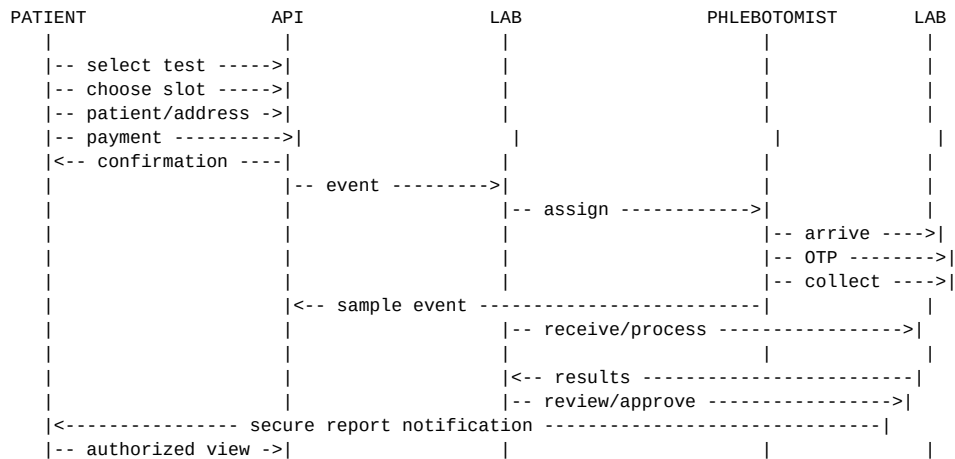
```
POST /api/v1/auth/login
↓ validate
↓ find user by normalized email/phone
↓ verify password
↓ load roles + tenant
↓ create session/refresh record
↓ set secure HTTP-only cookie
↓ return safe user + tenant context
```

```
Every later request:
cookie/session
↓
authenticate
↓
load user
↓
derive tenant context
↓
authorize permission
↓
domain controller
```

Acceptance criteria

- A lab admin can sign in.
- A suspended user cannot sign in.
- A receptionist cannot call report:publish.
- A user from Lab A cannot retrieve Lab B patient/order/report data.
- Logout invalidates the session.
- Expired/revoked sessions fail.
- All high-risk authentication actions are rate-limited and audited.

23. Booking-to-Report Sequence



This sequence reflects the source blueprint's complete lifecycle: patient booking/payment, lab assignment, collection/OTP, sample receipt/processing, authorized review, report generation and secure patient delivery.

filecite turn0file0 L92-L108

24. Failure & Recovery Design

Failure	System response
Payment succeeds but booking confirmation fails	Reconcile provider webhook; make booking/payment operation idempotent; show pending state.
Phlebotomist misses visit	Mark failed; notify staff; allow reassignment/reschedule.
OTP abuse	Expiry + attempt limit + IP/device/phone throttling where appropriate.
Sample rejected	Structured rejection reason + recollection task + audit event.
Report generation fails	Queue retry; retain report status; expose failure to authorized staff.
SMS provider outage	Retry queue; record delivery failure; do not block core clinical workflow.
Duplicate webhook	Idempotency key/provider event ID prevents duplicate mutation.
Tenant data leak attempt	Tenant-scoped repository returns no cross-tenant resource; security test covers it.
Incorrect finalized report	Amendment/reissue workflow; preserve versions and audit history.
Database outage	Managed DB recovery + backups + operational runbook; user-facing degraded mode.

The source blueprint highlights wrong patient/sample, incorrect results, sample rejection, missed collection, transport delays, tenant leakage, unauthorized report access, staff misuse and vendor outages as core risks.

filecite turn0file0L380-L391

25. AI — Later, Not First

AI should be added after the core workflow is reliable. The source blueprint proposes patient assistance, report explanation, operations analytics, delay analysis, demand forecasting and route optimization, with explicit guardrails.

■filecite■turn0file0■L399-L409■

Feature	Safe architecture
Patient assistant	Retrieval from lab-approved content; no invented medical instructions.
Report explanation	Read-only explanation layer; clearly not diagnosis; no result mutation.
Operations assistant	Tenant-scoped read-only tool access with permission checks.
Delay analysis	Use auditable event timestamps.
Demand forecasting	Planning support, not deterministic promises.
Route optimization	Recommendation only; human override.

Never connect an LLM directly to MongoDB with unrestricted write permissions.

26. Healthcare / Privacy Readiness

The uploaded blueprint intentionally does not prescribe a complete legal compliance model. It says the actual operating model should be reviewed for applicable Indian healthcare, privacy, contractual and regulatory requirements before launch, and that consent/security requirements should be designed in if interoperability ecosystems are used.

■filecite■turn0file0■L361-L364■

Engineering controls to prepare for that review

- Data classification for patient identity, contact information, clinical results and reports.
- Least-privilege RBAC.
- Tenant isolation tests.
- Immutable/high-integrity audit trail for sensitive actions.
- Encryption in transit and at rest through managed infrastructure.
- Private report storage and authorized retrieval.
- Backup/restore drills.
- Retention/deletion policy implemented as a configurable business/legal policy rather than ad-hoc deletion.
- Documented incident response and access review.
- Vendor contracts and data-processing responsibilities reviewed before production use.

Do not market the platform as “compliant” until the actual deployment, contracts, processes and applicable requirements have been reviewed.

27. Pre-Launch Checklist

Area	Must pass before pilot
Product	Patient booking; lab assignment/reschedule; OTP collection; sample tracking; authorized report approval; secure report view
Security	Tenant isolation; RBAC; report access control; OTP rate limiting; secrets; HTTPS; backups; audit logs
Operations	Sample rejection; failed collection; payment failure; notification retry/failure visibility; support investigation; report correction
Business	Pricing; subscription/contract model; lab onboarding; support; data ownership; privacy/legal review
Reliability	Health checks; logs; alerts; queue failure handling; backup restore test
Pilot	One real lab completes booking-to-report loop using realistic test cases

This consolidates the source blueprint's pre-launch checklist. ■filecite■turn0file0■L506-L547■

28. Final Build Blueprint

Build in this exact order

- 1 **Foundation:** monorepo, TypeScript, CI/CD, env validation, logging, error handling.
- 2 **Security core:** authentication, sessions, tenant context, RBAC, audit.
- 3 **Tenant setup:** labs, branches, settings, staff.
- 4 **Catalogue:** tests, parameters, packages, tenant-specific pricing.
- 5 **Patient:** patient records, addresses, family relationships.
- 6 **Booking:** order, appointment slots, checkout/payment abstraction.
- 7 **Collection:** assignment, phlebotomist mobile UI, OTP and collection events.
- 8 **Samples:** specimen IDs, events, receipt, rejection/recollection.
- 9 **Laboratory:** processing, results, validation and reviewer workflow.
- 10 **Reports:** PDF generation, private storage, versioning, approval and secure access.
- 11 **Async:** Redis/BullMQ notifications, reminders, report jobs.
- 12 **Commercial:** payments, invoices, coupons, branches, subscriptions.
- 13 **Operations:** dashboards, support, analytics, audit.
- 14 **Scale:** caching, CDN, horizontal scaling, workload extraction only when justified.
- 15 **Advanced:** LIS/instrument integration, interoperability, accounting, route planning and AI.

This preserves the uploaded blueprint's central architecture: multi-tenant healthcare SaaS with Patient Portal, Lab Admin, Phlebotomist App and supporting technician/pathologist/Super Admin workflows; MERN/modular monolith; MongoDB; secure object storage; queues; provider abstraction; and progressive scalability. ■filecite■turn0file0■L548-L564■

NEXT STEP: start Sprint 0 and Sprint 1. Do not begin with the patient booking UI. Build tenant/auth/RBAC and the database/domain foundation first, then implement the booking-to-report vertical slice.